

Homework 4

Due Date: [Check Google Classroom](#)

This exercise will give you an opportunity to learn how MIPS assembly language programs use heap memory. As we know, heap is an area of memory that is dynamically allocated at run time by programs. Languages like C and C++ provide functions in the standard library to allocate and release heap memory. In assembly language, the programmer has to manage the heap.

Problem

You want to write a program that obtains a set of floating-point (single-precision) values from user, sorts those values, and prints the sorted array on console or screen. We must know how many values the user wants to provide. There are two methods to determine the number of values.

The first one is to ask the user to provide a count of the values, followed by the actual values. Another method is to keep reading the values till a sentinel value is entered. We will adopt the first method. The program will prompt the user to enter the number of floating-point values to process. Once the number is known, we proceed to allocate memory from heap, and then start reading the actual values entered by the user.

When the user enters the count of values to process, we know how much heap memory is required, since each single-precision value requires 4 bytes, we need $4n$ bytes of heap memory for n values.

The next step is to allocate this much memory from heap. MARS provides a system call to accomplish this. The system call service number is 9 and the system call is called `sbrk`. Looking up the details in MARS help, we find that this service requires one parameter (in `$a0` register). That parameter is the number of bytes to allocate. The service returns the address of the allocated memory block (of the requested size) in `$v0` register. The address returned by the service, for example `0x10040000`, is the starting address of the memory block. The first single-precision value entered by the user will be stored at addresses `0x10040000`, `0x10040001`, `0x10040002`, and `0x10040003`. The next value entered by the user will be stored at addresses `0x10040004`, `0x10040005`, `0x10040006`, and `0x10040007`. **Keep in mind the values entered by user will not automatically get stored at the above-mentioned addresses.** It is your job as a programmer to store each value at the correct address. The service to read a single-precision value returns the value entered from keyboard in `$f0` register. It is your job to save the first value starting at the address `0x10040000`, and the next one starting at the address `0x10040004`, and so on. You know you have to run a loop for this purpose. Inside the loop you must increment the addresses properly so that no previously saved value is over-written.

Exercise 1

Write a MIPS assembly language function `getFloatArray`. This function requires two parameters. The first parameter is the number of floating-point values to obtain from keyboard. The second parameter is the address of the prompt string to print before getting each value. This function returns the address of the dynamically allocated memory block containing the values to the caller. You know which register is used to return a single value.

Exercise 2

Write a MIPS assembly language function `sortFloatArray`. This function takes two parameters. The first one is the number of elements in the array, and the second one is the starting address of the array. It sorts the values in ascending order. This function does not allocate any new memory. It sorts the array by swapping values.

Exercise 3

Write a MIPS assembly language function `printFloatArray`. This function takes two parameters. The first one is the number of elements in the array, and the second one is the starting address of the array. It prints each value on a new line.

Submission Instructions

Submit the following files.

1. `<rollnumber>-h4-ex1.asm` containing solution of exercise 1
2. `<rollnumber>-h4-ex2.asm` containing solution of exercise 2
3. `<rollnumber>-h4-ex3.asm` containing solution of exercise 3

You are allowed to use the code written for an exercise in another exercise. However, you must submit three separate files so that we can easily mark your submission. **Strictly follow the above naming convention.** Your submission shall not be marked otherwise.